

20. Microservices với Spring Boot

20.1. Giới thiệu về Microservices

Microservices là một kiến trúc phát triển phần mềm, trong đó một ứng dụng lớn được xây dựng như một tập hợp các dịch vụ nhỏ, độc lập, có thể triển khai độc lập, có thể mở rộng độc lập và giao tiếp với nhau thông qua các giao thức nhẹ (thường là HTTP/REST hoặc message queues).

Kiến trúc này đối lập với kiến trúc Monolithic truyền thống, nơi toàn bộ ứng dụng được xây dựng như một khối duy nhất, chặt chẽ.

20.1.1. Ưu điểm của Microservices

- **Khả năng mở rộng độc lập (Independent Scalability):** Mỗi dịch vụ có thể được mở rộng (scale up/out) độc lập dựa trên nhu cầu, tối ưu hóa việc sử dụng tài nguyên.
- **Triển khai độc lập (Independent Deployment):** Các dịch vụ có thể được triển khai, cập nhật mà không ảnh hưởng đến các dịch vụ khác, giúp tăng tốc độ phát hành.
- **Khả năng chịu lỗi (Resilience):** Lỗi trong một dịch vụ ít có khả năng ảnh hưởng đến toàn bộ hệ thống. Các dịch vụ có thể được cô lập và phục hồi nhanh chóng.
- **Linh hoạt công nghệ (Technology Heterogeneity):** Mỗi dịch vụ có thể được phát triển bằng các ngôn ngữ lập trình, framework, và cơ sở dữ liệu khác nhau, cho phép lựa chọn công nghệ phù hợp nhất cho từng tác vụ.
- **Dễ quản lý (Manageability):** Các codebase nhỏ hơn, dễ hiểu và quản lý hơn cho các nhóm phát triển nhỏ.
- **Tái sử dụng (Reusability):** Các dịch vụ có thể được tái sử dụng trong các ứng dụng hoặc dự án khác.

20.1.2. Nhược điểm của Microservices

- **Độ phức tạp (Complexity):** Tăng độ phức tạp trong việc quản lý, triển khai, giám sát và gỡ lỗi nhiều dịch vụ.

- **Phân tán dữ liệu (Distributed Data Management):** Quản lý dữ liệu nhất quán trên nhiều cơ sở dữ liệu độc lập là một thách thức.
- **Giao tiếp giữa các dịch vụ (Inter-service Communication):** Cần cơ chế giao tiếp hiệu quả và đáng tin cậy (ví dụ: API Gateway, Message Broker).
- **Giám sát và Logging (Monitoring & Logging):** Yêu cầu các công cụ giám sát và logging tập trung để theo dõi trạng thái của toàn bộ hệ thống.
- **Kiểm thử (Testing):** Kiểm thử các hệ thống phân tán phức tạp hơn nhiều so với ứng dụng monolithic.

20.2. Spring Boot và Microservices

Spring Boot là một framework lý tưởng để xây dựng microservices nhờ các tính năng sau:

- **Tự động cấu hình (Auto-configuration):** Giảm thiểu cấu hình, giúp nhanh chóng khởi tạo các dịch vụ.
- **Starters:** Cung cấp các dependency cần thiết cho các tính năng cụ thể (web, data, security, v.v.), đơn giản hóa việc quản lý thư viện.
- **Máy chủ nhúng (Embedded Servers):** Cho phép đóng gói ứng dụng thành một file JAR thực thi, dễ dàng triển khai.
- **Actuator:** Cung cấp các endpoint để giám sát và quản lý dịch vụ trong môi trường sản xuất.
- **Spring Cloud:** Một bộ công cụ toàn diện để xây dựng các hệ thống microservices phân tán.

20.3. Các thành phần chính trong kiến trúc Microservices với Spring Boot

Để xây dựng một hệ thống microservices hoàn chỉnh, ngoài các dịch vụ lõi được xây dựng bằng Spring Boot, chúng ta thường cần các thành phần hỗ trợ khác:

20.3.1. Service Discovery (Phát hiện dịch vụ)

Trong một hệ thống microservices, các dịch vụ cần tìm và giao tiếp với nhau. Service Discovery cho phép các dịch vụ đăng ký vị trí của chúng và các dịch vụ khác có thể tìm thấy chúng. Các giải pháp phổ biến:

- **Eureka (Netflix OSS):** Một máy chủ phát hiện dịch vụ dựa trên REST.
- **Consul:** Một công cụ phân tán để khám phá dịch vụ, cấu hình và điều phối.
- **Zookeeper:** Một dịch vụ tập trung để duy trì thông tin cấu hình, đặt tên, cung cấp đồng bộ hóa phân tán và các dịch vụ nhóm.

20.3.2. API Gateway

API Gateway là một điểm truy cập duy nhất cho tất cả các client (web, mobile) để tương tác với các microservices. Nó xử lý các tác vụ như định tuyến request, cân bằng tải, xác thực/ủy quyền, giới hạn tốc độ, và chuyển đổi giao thức.

- **Spring Cloud Gateway:** Một API Gateway dựa trên Spring Framework 5, Spring Boot 2 và Project Reactor, cung cấp hiệu suất cao và khả năng cấu hình linh hoạt.
- **Zuul (Netflix OSS):** Một API Gateway dựa trên JVM, cung cấp các tính năng định tuyến, lọc và bảo mật.

20.3.3. Load Balancing (Cân bằng tải)

Khi có nhiều instance của một dịch vụ, cân bằng tải giúp phân phối request đồng đều giữa các instance đó để tối ưu hóa hiệu suất và khả năng chịu lỗi.

- **Ribbon (Netflix OSS):** Một thư viện cân bằng tải phía client, tích hợp với Eureka.
- **Spring Cloud LoadBalancer:** Một giải pháp cân bằng tải phía client thay thế cho Ribbon, được xây dựng trên Spring WebFlux.

20.3.4. Distributed Configuration (Cấu hình phân tán)

Trong môi trường microservices, việc quản lý cấu hình cho hàng chục hoặc hàng trăm dịch vụ có thể rất phức tạp. Distributed Configuration Server cung cấp một nơi tập trung để quản

lý cấu hình cho tất cả các dịch vụ.

- **Spring Cloud Config Server:** Một máy chủ cấu hình tập trung, hỗ trợ Git, Subversion, và các backend khác.

20.3.5. Circuit Breaker (Ngắt mạch)

Circuit Breaker là một mẫu thiết kế để ngăn chặn lỗi lan truyền trong hệ thống phân tán. Khi một dịch vụ gặp sự cố, Circuit Breaker sẽ ngăn các request tiếp theo đến dịch vụ đó, thay vào đó trả về một phản hồi dự phòng hoặc lỗi ngay lập tức, giúp hệ thống phục hồi và tránh quá tải.

- **Resilience4j:** Một thư viện nhẹ, dễ sử dụng cho Circuit Breaker, Rate Limiter, Retry, Bulkhead, và TimeLimiter.
- **Hystrix (Netflix OSS):** Một thư viện Circuit Breaker phổ biến (hiện đã ở chế độ bảo trì).

20.3.6. Distributed Tracing (Truy vết phân tán)

Trong một hệ thống microservices, một request có thể đi qua nhiều dịch vụ khác nhau. Distributed Tracing giúp theo dõi luồng của một request qua toàn bộ hệ thống, giúp gỡ lỗi và phân tích hiệu suất.

- **Spring Cloud Sleuth:** Tích hợp các thư viện tracing như Zipkin hoặc OpenTelemetry vào ứng dụng Spring Boot.
- **Zipkin:** Một hệ thống tracing phân tán để thu thập và hiển thị dữ liệu thời gian của các request.

20.3.7. Message Broker (Hàng đợi tin nhắn)

Message Broker được sử dụng để giao tiếp bất đồng bộ giữa các dịch vụ, giúp giảm sự phụ thuộc lẫn nhau và tăng khả năng chịu lỗi.

- **RabbitMQ:** Một message broker phổ biến, hỗ trợ AMQP.
- **Apache Kafka:** Một nền tảng streaming phân tán, lý tưởng cho việc xử lý dữ liệu lớn và real-time.

20.4. Ví dụ đơn giản về Microservices với Spring Boot

Để minh họa, chúng ta sẽ xem xét một ví dụ đơn giản với hai microservice: `Product Service` và `Order Service`, sử dụng Eureka cho Service Discovery.

20.4.1. Eureka Server

pom.xml:

XML

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Application Class:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml:

YAML

```
server:
  port: 8761
eureka:
  client:
```

```
register-with-eureka: false
fetch-registry: false
```

20.4.2. Product Service

pom.xml:

XML

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Application Class:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

@SpringBootApplication
@EnableEurekaClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

application.yml:

YAML

```
spring:
  application:
    name: product-service
server:
  port: 8081
```

```
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
```

Product Controller (ví dụ):

Java

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable Long id) {
        return "Product " + id + " from Product Service";
    }
}
```

20.4.3. Order Service

pom.xml:

XML

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId> <!-- Để gọi
Product Service -->
</dependency>
```

Application Class:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;

@SpringBootApplication
@EnableEurekaClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

application.yml:

YAML

```
spring:
  application:
    name: order-service
server:
  port: 8082
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
```

Feign Client để gọi Product Service:

Java

```
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "product-service")
public interface ProductServiceClient {

    @GetMapping("/products/{id}")
```



```
String getProduct(@PathVariable("id") Long id);  
}
```

Order Controller:

Java

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
@RequestMapping("/orders")  
public class OrderController {  
  
    private final ProductServiceClient productServiceClient;  
  
    public OrderController(ProductServiceClient productServiceClient) {  
        this.productServiceClient = productServiceClient;  
    }  
  
    @GetMapping("/{orderId}/product/{productId}")  
    public String getOrderDetails(@PathVariable Long orderId, @PathVariable  
Long productId) {  
        String productInfo = productServiceClient.getProduct(productId);  
        return "Order " + orderId + " details: " + productInfo;  
    }  
}
```

20.5. Bản chất của Microservices với Spring Boot

Bản chất của việc xây dựng Microservices với Spring Boot là tận dụng các tính năng mạnh mẽ của Spring Boot (như tự động cấu hình, Starters, máy chủ nhúng) để đơn giản hóa việc phát triển từng dịch vụ nhỏ, độc lập. Sau đó, sử dụng các dự án con của Spring Cloud (như Eureka, Feign, Gateway, Config, Resilience4j) để giải quyết các thách thức cố hữu của kiến trúc phân tán như phát hiện dịch vụ, giao tiếp giữa các dịch vụ, quản lý cấu hình, cân bằng tải, và khả năng chịu lỗi. Mục tiêu cuối cùng là xây dựng một hệ thống linh hoạt, có khả năng mở rộng cao, dễ bảo trì và có thể triển khai độc lập, phù hợp với các yêu cầu của các ứng dụng hiện đại.